

Yocto SELinux

目录

About SELinux

SELinux Mode

Yocto SELinux 开发环境搭建

Enable SELinux

自定义 Sepolicy

Debug Log

audit2allow 工具介绍

常用 debug 指令

Android SELinux & Yocto SELinux 对比

Author: Zpengz

About SELinux

DAC（Discretionary Access Control）：传统的 Linux 权限控制方式，基于用户和组的权限。文件拥有者可以自由修改文件的访问权限。

SELinux（Security-Enhanced Linux）：基于强制访问控制（MAC），通过安全策略决定资源访问，权限不可由任意用户随意更改。

对比项	DAC (传统 Linux 权限)	SELinux (MAC)
权限控制方式	用户/组	安全策略/标签
灵活性	高	低（更安全）
安全性	较弱	强
管理粒度	粗	细

SELinux 给每个进程和文件都分配标签，提供精细化的管控，将每个进程拆分成独立的域并设定独有的权限。**SELinux** 策略规则定义了进程如何与文件交互，以及进程如何相互交互。只有存在明确允许的 **SELinux** 策略规则时，才能允许访问，降低了被恶意攻击的风险。和 **Android** 类似，当前也是采取 **DAC+SELinux** 的管控模式。

Apparmor 也是基于强制访问控制（MAC），之前的方案是使用 **Apparmor+DAC**，**Apparmor** 和 **SELinux** 都是基于 MAC 的管控策略，但是各有一些优缺点：

特性	SELinux	AppArmor
策略类型	标签/类型强制	路径强制
粒度	细（基于进程/文件标签）	较粗（文件路径级）
配置复杂度	中	低
debug 灵活性	中等(可单编 selinux, push 文件验证)	高(可动态配置 sepolicy)

SELinux Mode

SELinux 可使用三种模式之一运行：enforcing（强制）、permissive（宽容）或 disabled（禁用）。

- Enforcing 模式是默认操作模式，在 enforcing 模式下 SELinux 可正常运行，并在整个系统中强制实施载入的安全策略。
- 在 permissive 模式中，系统会象 enforcing 模式一样加载安全策略，包括标记对象并在日志中记录访问拒绝条目，但它并不会拒绝任何操作。不建议在生产环境系统使用 permissive 模式，但 permissive 模式对 SELinux 策略开发和调试很有帮助。
- 强烈建议不要使用禁用（disabled）模式，它不仅会使系统避免强制使用 SELinux 策略，还会避免为任何持久对象（如文件）添加标签，这使得在以后启用 SELinux 非常困难。

同 Android SELinux 一样，在 debug 阶段针对 SELinux Mode 也可以使用 setenforce/getenforce 指令自行切换：

```

root@auto8676p164uostbox: #
root@auto8676p164uostbox: # getenforce
Enforcing
root@auto8676p164uostbox: #
root@auto8676p164uostbox: # setenforce 0
root@auto8676p164uostbox: # getenforce
Permissive
root@auto8676p164uostbox: #
root@auto8676p164uostbox: #
root@auto8676p164uostbox: # setenforce 1
root@auto8676p164uostbox: # getenforce
Enforcing
root@auto8676p164uostbox: #
root@auto8676p164uostbox: #

```

Yocto SELinux 开发环境搭建：

- 1) 构建 Yocto SELinux 必须集成 meta-selinux 仓库，建议首先从开源库中将这个 repo 拉下来, 参考指令:

git clone <https://git.yoctoproject.org/meta-selinux>

****download 时请注意要和 Yocto 版本对应，版本不匹配会导致后续编译报错****

- 2) 集成 chcon，在 SELinux 初始化阶段会检查是否有集成 chcon，未安装 chcon 会导致 init 报错，无法正常 enable SELinux; 集成 chcon 可参考以下修改：

11592383 [ALPS10458450] busybox: add selinux support ☒ recipes-core/busybox/busybox_%bbappend()

FILE

```

1 FILESEXTRAPATHS:append := ":${THISDIR}/files"
2
3 # remove syslog.cfg for disable syslog
4 SRC_URI:remove = "file://syslog.cfg"
5 SRC_URI:append = "file:///disable-syslog-klog.cfg"

```

FILE

```

1 FILESEXTRAPATHS:append := ":${THISDIR}/files"
2
3 DEPENDS += "${@bb.utils.contains('DISTRO_FEATURES', 'selinux', 'libselinux', '', d)}"
4
5 # remove syslog.cfg for disable syslog
6 SRC_URI:remove = "file://syslog.cfg"
7 SRC_URI:append = "file:///disable-syslog-klog.cfg"
8 SRC_URI:append = "${@bb.utils.contains('DISTRO_FEATURES', 'selinux', 'file:///enable-selinux.cfg',
9                                     '', d)}"

```

11592383 [ALPS10458450] busybox: add selinux support ☒ recipes-core/busybox/files/enabled-selinux.cfg

new file mode regular (100644)

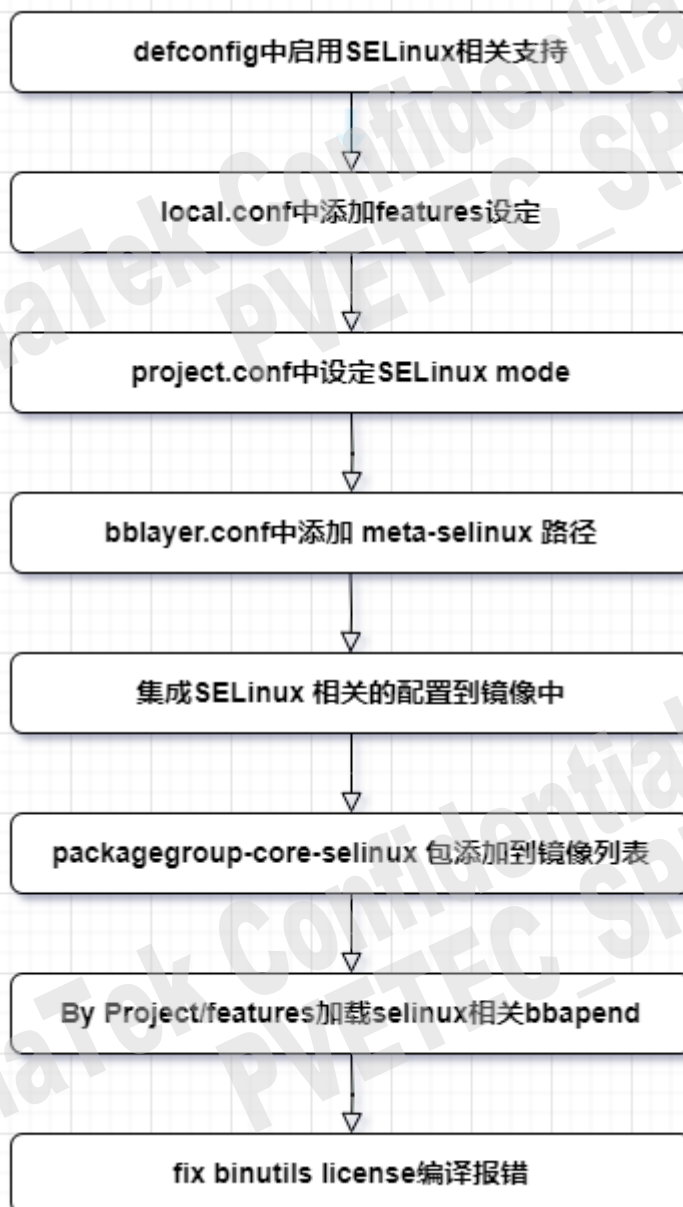
FILE

```

1 CONFIG_SELINUX=y
2 CONFIG_FEATURE_TAR_SELINUX=y
3 CONFIG_CHCON=y
4 CONFIG_SETENFORCE=y
5 CONFIG_GETSEBOOL=y
6 CONFIG_LOAD_POLICY=y
7 CONFIG_MATCHPATHCON=y
8 CONFIG_RUNCON=y
9 CONFIG_SELINUXENABLED=y
10 CONFIG_SESTATUS=y
11 CONFIG_SETENFORCE=y
12 CONFIG_SETFILES=y
13 CONFIG_FEATURE_SETFILES_CHECK_OPTION=y
14 CONFIG_RESTORECON=y
15 CONFIG_GETSEBOOL=y
16

```

Enable SELinux



1) 在 defconfig 中启用 SELinux 相关支持

11582886 [ALPS10458450] 8676_selinux project enable selinux ☒ recipes-kernel/linux/linux-mtk-extension-uos/selinux_uos.config

Base Patchset 6 Download Show Outline Show I

new file mode executable (100755)

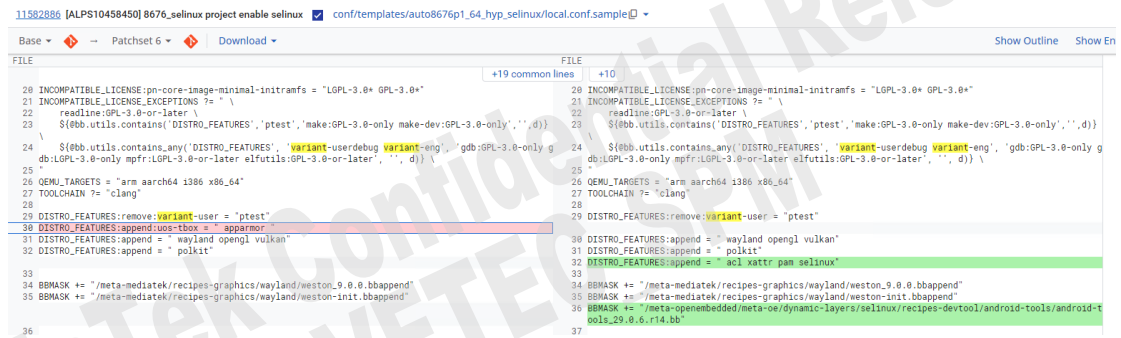
```

FILE
1 # CONFIG_SECURITY_APPARMOR is not set
2 # CONFIG_DEFAULT_SECURITY_APPARMOR is not set
3 CONFIG_AUDIT=y
4 CONFIG_SECURITY=y
5 CONFIG_SECURITYFS=y
6 CONFIG_SECURITY_NETWORK=y
7 CONFIG_SECURITY_SELINUX=y
8 CONFIG_SECURITY_SELINUX_BOOTPARAM=y
9 CONFIG_SECURITY_SELINUX_DEVELOP=y
10 CONFIG_SECURITY_SELINUX_AVC_STATS=y
11 CONFIG_SECURITY_SELINUX_CHECKREQPROT_VALUE=1
12 CONFIG_EXT4_FS=y
13 CONFIG_EXT4_FS_POSIX_ACL=y
14 CONFIG_EXT4_FS_SECURITY=y
15
16

```

****Apparmor 和 SELinux 不能同时开启，如果配置中有打开 apparmor 则需要注释掉

2) local.conf 中添加 features 设定



在 Yocto 项目中，**acl**、**xattr**、**pam** 和 **selinux** 这几个 DISTRO_FEATURES 之间有一定的依赖和关联。：

启用文件系统的访问控制列表（Access Control List）支持。**ACL** 允许对文件和目录设置更细粒度的权限控制。

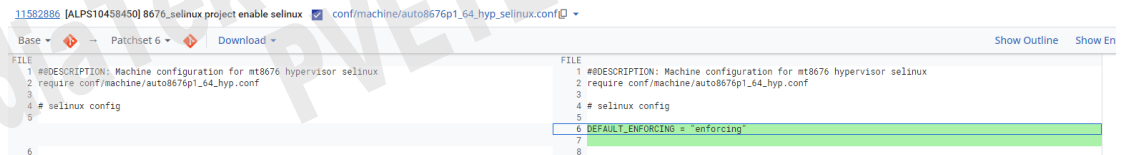
启用文件系统的扩展属性（eXtended ATTRibutes）支持。**xattr** 允许为文件和目录存储额外的元数据，SELinux 就是通过 xattr 存储安全上下文的。

启用可插拔认证模块（Pluggable Authentication Modules）支持。**PAM** 是 Linux 下用户认证的标准框架，很多安全机制（包括 SELinux）都可以通过 PAM 集成到系统认证流程中。

启用 **SELinux**（Security-Enhanced Linux）强制访问控制系统

*****关于 BBMASK 的设定是为了修复编译报错，将相关内容添加到黑名单*****

3) project.conf 中设定 SELinux mode



如需更改 code 中的 SELinux Mode 设定，可参考上述修改

4) bblayer.conf 中添加 meta-selinux 路径，确保 meta-selinux 在构建层次中，并且优先级合适



5) by feature 集成与 SELinux 相关的配置和文件到最终生成的镜像中

11613876 [ALPS10458450] build selinux image[1/2] ☒ recipes-auto/images/mtk-core-image-auto8676.bb ☐	
Base ☐ Patchset 3 ☐ Download ☐	Show Outline Show En
FILE	FILE
+8 common lines	
9 LIC_FILES_CHKSUM = "file://\$COMMON_LICENSE_DIR/MIT;md5=0835ade698ebcf8506ecda2f7b4f302"	9 LIC_FILES_CHKSUM = "file://\$COMMON_LICENSE_DIR/MIT;md5=0835ade698ebcf8506ecda2f7b4f302"
10	10
11 EXTRA_IMAGECMD:ext4 = '-i 4096 -b 4096 -O 'has_journal'	11 EXTRA_IMAGECMD:ext4 = '-i 4096 -b 4096 -O 'has_journal'
12	12
13 inherit core-image	13 inherit core-image
14	14
15 inherit mkusurdata	15 inherit mkusurdata
16 inherit staging-copyfile	16 inherit staging-copyfile
17 inherit debug-level	17 inherit debug-level
18 inherit journal-log	18 inherit journal-log
19 python () {	19 python () {
20 if 'selinux' in d.getVar('DISTRO_FEATURES').split():	20 if 'selinux' in d.getVar('DISTRO_FEATURES').split():
21	21

6) by feature 向镜像安装包列表中添加 packagegroup-core-selinux 包

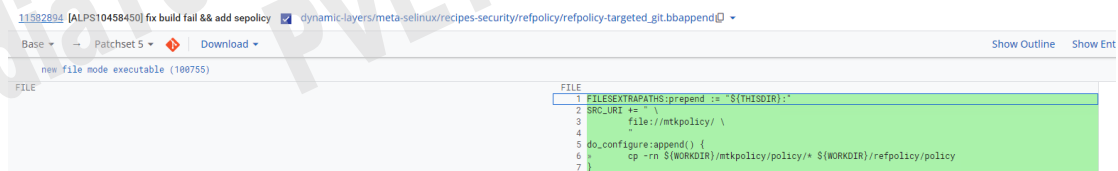
11540045 auto8676p1_64_hyp_selinux enable selinux[1/2] ☒ recipes-auto/images/mtk-core-image-auto8676-uos.bb ☐	
Base ☐ Patchset 2 ☐ Download ☐	Show Outline Show Entire File
FILE	FILE
+44 common lines	
1 DESCRIPTION = "core image for mt8676 uos tb0x & apa"	1 DESCRIPTION = "core image for mt8676 uos tb0x & apa"
2	2
3 LICENSE = "MIT"	3 LICENSE = "MIT"
4 LIC_FILES_CHKSUM = "file://\$COMMON_LICENSE_DIR/MIT;md5=0835ade698ebcf8506ecda2f7b4f302"	4 LIC_FILES_CHKSUM = "file://\$COMMON_LICENSE_DIR/MIT;md5=0835ade698ebcf8506ecda2f7b4f302"
5	5
6 EXTRA_IMAGECMD:ext4 = '-i 4096 -b 4096 -O 'has_journal'	6 EXTRA_IMAGECMD:ext4 = '-i 4096 -b 4096 -O 'has_journal'
7	7
8 inherit core-image	8 inherit core-image
9 inherit mkusurdata	9 inherit mkusurdata
10 inherit debug-level	10 inherit debug-level
11 inherit journal-log	11 inherit journal-log
12	12
13 # only uos tb0x required package, add in uos-tbox-image-install.inc	13 # only uos tb0x required package, add in uos-tbox-image-install.inc
14 require uos-tbox-image-install.inc	14 require uos-tbox-image-install.inc
15	15
16 IMAGE_FEATURES += "read-only-rootfs"	16 IMAGE_FEATURES += "read-only-rootfs"
17	17
18 IMAGE_INSTALL = "	18 IMAGE_INSTALL = "
19 packagegroup-core-boot \	19 packagegroup-core-boot \
20 coreutils \	20 coreutils \
21 android-tools-adb \	21 android-tools-adb \
22	22
+10	
23	23
24	24
25	25
26	26
27	27
28	28
29	29
30	30
31	31
32	32
33	33
34	34
35	35
36	36
37	37
38	38
39	39
40	40
41	41
42	42
43	43
44	44
45	45
46	46
47	47
48	48
49	49
50	50
51	51
52	52
53	53
54	54
55	55
56	56
57	57
58	58
59	59
60	60
61	61
62	62
63	63
64	64
65	65
66	66
67	67
68	68
69	69
70	70
71	71
72	72
73	73
74	74
75	75
76	76
77	77
78	78
79	79
80	80
81	81
82	82
83	83
84	84
85	85
86	86
87	87
88	88
89	89
90	90
91	91
92	92
93	93
94	94
95	95
96	96
97	97
98	98
99	99
100	100
101	101
102	102
103	103
104	104
105	105
106	106
107	107
108	108
109	109
110	110
111	111
112	112
113	113
114	114
115	115
116	116
117	117
118	118
119	119
120	120
121	121
122	122
123	123
124	124
125	125
126	126
127	127
128	128
129	129
130	130
131	131
132	132
133	133
134	134
135	135
136	136
137	137
138	138
139	139
140	140
141	141
142	142
143	143
144	144
145	145
146	146
147	147
148	148
149	149
150	150
151	151
152	152
153	153
154	154
155	155
156	156
157	157
158	158
159	159
160	160
161	161
162	162
163	163
164	164
165	165
166	166
167	167
168	168
169	169
170	170
171	171
172	172
173	173
174	174
175	175
176	176
177	177
178	178
179	179
180	180
181	181
182	182
183	183
184	184
185	185
186	186
187	187
188	188
189	189
190	190
191	191
192	192
193	193
194	194
195	195
196	196
197	197
198	198
199	199
200	200
201	201
202	202
203	203
204	204
205	205
206	206
207	207
208	208
209	209
210	210
211	211
212	212
213	213
214	214
215	215
216	216
217	217
218	218
219	219
220	220
221	221
222	222
223	223
224	224
225	225
226	226
227	227
228	228
229	229
230	230
231	231
232	232
233	233
234	234
235	235
236	236
237	237
238	238
239	239
240	240
241	241
242	242
243	243
244	244
245	245
246	246
247	247
248	248
249	249
250	250
251	251
252	252
253	253
254	254
255	255
256	256
257	257
258	258
259	259
260	260
261	261
262	262
263	263
264	264
265	265
266	266
267	267
268	268
269	269
270	270
271	271
272	272
273	273
274	274
275	275
276	276
277	277
278	278
279	279
280	280
281	281
282	282
283	283
284	284
285	285
286	286
287	287
288	288
289	289
290	290
291	291
292	292
293	293
294	294
295	295
296	296
297	297
298	298
299	299
300	300
301	301
302	302
303	303
304	304
305	305
306	306
307	307
308	308
309	309
310	310
311	311
312	312
313	313
314	314
315	315
316	316
317	317
318	318
319	319
320	320
321	321
322	322
323	323
324	324
325	325
326	326
327	327
328	328
329	329
330	330
331	331
332	332
333	333
334	334
335	335
336	336
337	337
338	338
339	339
340	340
341	341
342	342
343	343
344	344
345	345
346	346
347	347
348	348
349	349
350	350
351	351
352	352
353	353
354	354
355	355
356	356
357	357
358	358
359	359
360	360
361	361
362	362
363	363
364	364
365	365
366	366
367	367
368	368
369	369
370	370
371	371
372	372
373	373
374	374
375	375
376	376
377	377
378	378
379	379
380	380
381	381
382	382
383	383
384	384
385	385
386	386
387	387
388	388
389	389
390	390
391	391
392	392
393	393
394	394
395	395
396	396
397	397
398	398
399	399
400	400
401	401
402	402
403	403
404	404
405	405
406	406
407	407
408	408
409	409
410	410
411	411
412	412
413	413
414	414
415	415
416	416
417	417
418	418
419	419
420	420
421	421
422	422
423	423
424	424
425	425
426	426
427	427
428	428
429	429
430	430
431	431
432	432
433	433
434	434
435	435
436	436
437	437
438	438
439	439
440	440
441	441
442	442
443	443
444	444
445	445
446	446
447	447
448	448
449	449
450	450
451	451
452	452
453	453
454	454
455	455
456	456
457	457
458	458
459	459
460	460
461	461
462	462
463	463
464	464
465	465
466	466
467	467
468	468
469	469
470	470
471	471
472	472
473	473
474	474
475	475
476	476
477	477
478	478
479	479
480	480
481	481
482	482
483	483
484	484
485	485
486	486
487	487
488	488
489	489
490	490
491	491
492	492
493	493
494	494
495	495
496	496
497	497
498	498
499	499
500	500
501	501
502	502
503	503
504	504
505	505
506	506
507	507
508	508
509	509
510	510
511	511
512	512
513	513

自定义 Sepolicy:

sepolicy 存放在路径:

meta/meta-mediatek/dynamic-layers/meta-selinux/recipes-security/refpolicy/mtkpolicy/policy/modules/

相关修改为:



```
11582894 [ALPS10458450] fix build fail && add sepolity dynamic-layers/meta-selinux/recipes-security/refpolicy/refpolicy-targeted_git.bbappend
Base Patchset 5 Download Show Outline Show Ent
new file mode executable (100755)
FILE
1 FILESEXPATHS:prepend := "${THISDIR}:"
2 SRC_URI += " \
3 file://mtkpolicy/ \
4
5 do_configure:append() {
6 cp -rn ${WORKDIR}/mtkpolicy/policy/* ${WORKDIR}/refpolicy/policy
7 }
```

如何针对进程添加 sepolity:

在上述目录下存放的多个案例， 每个需要拆分的进程需自行创建三个文件， 比如是 netd.if netd.fc netd.te;

- te 文件：类型扩展，定义域和类型规则。
- if 文件：接口文件，定义可复用的策略接口。
- fc 文件：文件上下文，定义文件路径与 SELinux 类型的映射。

考虑到会有一些格式问题，建议是从已有案例中直接 copy 出来，操作步骤如下:

- 1)copy netd.if netd.fc netd.te 并将 copy 后的文件重命名， 命名为自己进程相关的， 比如 mtknetwork
- 2) 将打开重命名后的.fc 文件， 在其中设定当前进程的 bin 档路径与对应的 label
- 3).if 文件中如果不需要增加接口添加 sepolity 则无需修改
- 4) 主要配置在.te 文件中， sepolity 相关语法和需要注意的点请查看下面案例及说明

```

# mtknetworkmanager.te
# policy_module 名称必须与文件名一致
policy_module(mtknetworkmanager, 1.0)

# 声明进程 domain 和可执行文件的 type
type mtknetwork_t;
type mtknetworkmanager_exec_t;

# 声明 domain transition, 确保进程启动后进入 mtknetwork_t
init_daemon_domain(mtknetwork_t, mtknetworkmanager_exec_t)

# 引用外部 type、class, 必须在 gen_require 作用域中声明
gen_require(`
    type locale_t; # 需要访问的目标 type
    class file { read open getattr }; # 需要的文件操作权限
    class dir { search }; # 需要的目录操作权限
    class vsock_socket { listen }; # 需要的 socket 权限
`)

# 允许 mtknetwork_t 进程读取 locale_t 类型的文件
allow mtknetwork_t locale_t:file { read open getattr }; # 读取
locale 相关文件

# 允许 mtknetwork_t 进程搜索 locale_t 类型的目录
allow mtknetwork_t locale_t:dir search; # 搜索 locale 相关目录

# 允许 mtknetwork_t 进程监听 vsock_socket
allow mtknetwork_t self:vsock_socket { listen }; # 监听 vsock
socket

```

***** policy_module 名称必须与 .te 文件名一致，否则编译会报错。

***** type/typealias/attribute 声明所有用到的 type 必须先声明，否则编译会报错。type 和 class 以及 permission 声明后，才能在 allow 等规则中使用。

***** gen_require 的使用引用外部 type、attribute、class 时必须用 gen_require，否则编译器找不到符号。只需声明本 policy 文件中用到的外部符号，避免冗余。

***** domain transition，init_daemon_domain 等宏用于声明进程的 domain 切换 (transition)，否则进程不会进入你定义的 domain。transition 相关的 type 也要声明。

*****allow 规则必须明确 scontext、tcontext、tclass、permissions。权限要尽量最小化，只给进程实际需要的权限，避免过度授权，可以用 audit2allow 工具分析缺失的权限。

***** 每条规则建议加注释，便于后续维护和安全审计。

***** 尽量不要直接添加 unlabeled_t 相关的权限，这会导致添加权限的权限过大，可能会遇到 neverallow 限制导致编译报错

*****如果自己新建目录存放，请将 metadata.xml 拷贝过去，否则会编译报错*****

```

NOTE: recipe uos-tbox-repolicy-targeted-2.20240226+git-r0: task do_compile: Started
ERROR: uos-tbox-repolicy-targeted-2.20240226+git-r0 do_compile: oe_runmake failed
ERROR: uos-tbox-repolicy-targeted-2.20240226+git-r0 do_compile: ExecutionError/mfs/mhfsf58/ot909160/CAS_REAL/spm-dev-scarthgap_mp3_selinux-20250623_1500/merged/build/tmp/work/auto8676p1_64_hyp-poky-l
oe-uos-tbox-repolicy-targeted/2.20240226+git-temp/run.do_compile.7942', 1, None, None)
ERROR: Logfile of failure stored in: /mfs/mhfsf58/ot909160/CAS_REAL/spm-dev-scarthgap_mp3_selinux-20250623_1500/merged/build/tmp/work/auto8676p1_64_hyp-poky-linux-uos-tbox-uos-tbox-repolicy-targeted/2.
20240226+git-temp/log.do_compile.7942
log do_compile failed
DEBUG: Executing shell function do_compile
NOTE: make NAME=targeted TYPE=mc DISTRO=debian UBA=n UNK_PERS=allow DIRECT_INITRC=y SYSTEM=y MONOLITHIC= CUSTOM_BUILDSPC= QUBIC=y M.S_SENS=0 M.S_CATS=1024 MCS_CATS=1024 tc_urbindir=/mfs/mhfsf58/o
909160/CAS_REAL/spm-dev-scarthgap_mp3_selinux-20250623_1500/merged/build/tmp/work/auto8676p1_64_hyp-poky-linux-uos-tbox-uos-tbox-repolicy-targeted/2.20240226+git/recipe-sysroot-native/usr/bin OUTPUT_PATH
33 CC=gcc CFLAGS=-system/mfs/mhfsf58/ot909160/CAS_REAL/spm-dev-scarthgap_mp3_selinux-20250623_1500/merged/build/tmp/work/auto8676p1_64_hyp-poky-linux-uos-tbox-uos-tbox-repolicy-targeted/2.20240226+gi
targeted/2.20240226+git/recipe-sysroot-native/usr/bin/python3-native/python3
Generating interface templates into doc/tmp/iftemplates
ERROR: oe_runmake failed
WARNING: exit code 1 from a shell command.
NOTE: recipe uos-tbox-repolicy-targeted-2.20240226+git-r0: task do_compile: failed

```

Debug Log

- /var/log/audit/audit.log: SELinux 审计日志(当前默认会保存, debug 时最常用)
- dmesg: 内核拒绝、启动阶段, 通常适用于抓开机阶段的 avc denied 打印
- journalctl: 适用于抓用户空间服务、systemd 相关的日志及 avc denied 打印

除此之外, /data/debuglogger/下默认会抓 kernel log 以及 main log 等, 也可以直接查看该 log 日志。

audit2allow 工具介绍

audit2allow 是 SELinux 提供的一个辅助工具, 用于分析 SELinux 的审计日志 (通常是 /var/log/audit/audit.log), 自动生成相应的策略规则 (allow 语句), 帮助开发者快速定位和解决因策略不完善导致的访问拒绝 (AVC Denial) 问题。

使用 audit2allow 的方式有两种, 一种是在编译环境中进行, 另外一种是直接使用平台中编译出来的程序执行, 推荐使用后者。

- 1) 将抓取到的 log push 到平台中
- 2) 以默认存储的/var/log/audit/audit.log 为例, 使用 audit2allow -i /var/log/audit/audit.log 指令解析

```

#===== adbd_t =====
allow adbd_t bin_t:dir search;
allow adbd_t etc_runtime_t:file { open read };
allow adbd_t locale_t:file { getattr open };
allow adbd_t systemd_journal_t:dir { getattr open read search };
allow adbd_t systemd_journal_t:file { getattr map open read };
allow adbd_t systemd_journalctl_exec_t:file { execute execute_no_trans getattr map open read };
allow adbd_t unlabeled_t:dir { add_name search write };
allow adbd_t unlabeled_t:file { create getattr ioctl open write };
allow adbd_t unlabeled_t:link_file read;
allow adbd_t var_log_t:dir search;

#===== mobilelog_t =====
allow mobilelog_t unlabeled_t:dir search;
allow mobilelog_t unlabeled_t:link_file read;

```

以上述结果为例，如果存在权限缺失，会按进程生成相应的修改

*****audit2allow 工具不能解析整个文件夹，有整理两个脚本用于搜集 log 中的所有
avc denied 打印，生成的 summary 信息可以使用 audit2allow 工具来解析*****

常用 debug 指令

ps -eZ 指令用于检查进程的 label 是否符合预期，ex:

```
# ps -eZ |grep netd
```

```
system_u:system_r:netd_setup_t:s0 1836 ? 00:00:00 netd_setup.sh
```

```
system_u:system_r:netd_setup_t:s0 1840 ? 00:00:00 netd
```

busybox ls -Z /var/log/audit/audit.log 用于检查文件的 label，ls -Z 无法直接使用，ex:

```
# busybox ls -Z /var/log/audit/audit.log
```

```
system_u:object_r:auditd_log_t:s0 /var/log/audit/audit.log
```

sesearch 指令检查 policy 是否有 build in，ex:

```
# sesearch -A -s mtknetwork_t
```

```
allow mtknetwork_t initrc_t:fifo_file { append getattr ioctl lock read write };
```

```
allow mtknetwork_t initrc_t:process sigchld;
```

```
allow mtknetwork_t locale_t:dir search;
```

```
allow mtknetwork_t locale_t:file { getattr open read };
```

```
allow mtknetwork_t mtknetwork_t:association sendto;
```

```
allow mtknetwork_t mtknetwork_t:dir { add_name getattr ioctl lock open read
```

```
remove_name search write };
```

```
allow mtknetwork_t mtknetwork_t:file { append create getattr ioctl link lock open read
```

```
rename setattr unlink write };
```

```
allow mtknetwork_t mtknetwork_t:lnk_file { getattr ioctl lock read };
```

```
allow mtknetwork_t mtknetwork_t:lockdown { confidentiality integrity };
```

```
allow mtknetwork_t mtknetwork_t:process { fork sigchld };
```

chcon 指令临时更改文件的上下文设定，ex:

```
chcon -t httpd_sys_content_t /var/www/html/index.html
```

restorecon 指令恢复文件的默认 SELinux 安全上下文，ex:
restorecon -RV /path/to/file

Android SELinux & Yocto SELinux 对比

1) 文件类型对比

文件类型	Android SELinux	Yocto SELinux	说明
类型强制	*.te	*.te	基本语法相同(Yocto多了module声明与作用域引用声明)
文件上下文	file_contexts	*.fc	Android 无扩展名
接口定义	*.te (public/)	*.if	Yocto 专用接口文件(通常无需关注)
属性上下文	property_contexts	*.fc 中处理(Android独有)	Android 专有
服务上下文	service_contexts	同上	Android 专有
genfs 上下文	genfs_contexts	同上	Android 独立文件

2) 命名规范对比

规范	Android SELinux	Yocto SELinux
label定义规范	无强制后缀，如 my_app	后缀加 _t，如 my_app_t
contexts定义规范	无扩展名，直接写路径	Yocto 使用 gen_context() 宏，类型名通常带 _t 后缀

比如:

```
/system/bin/myservice u:object_r:my_service_exec:s0
/data/myapp(/.*)? u:object_r:my_app_data_file:s0
/dev/my_device u:object_r:my_device:s0
```

和

```
/usr/bin/myservice -- gen_context(system_u:object_r:my_service_exec_t,s0)
/var/lib/myapp(/.*)? gen_context(system_u:object_r:my_app_var_t,s0)
```

3) MLS 等安全级别对比

Security 级别对比	Android 现状	Yocto 现状
MLS检查	已开启	未开启
ioctlcmd	需单独配置	无需单独配置

4) 单编 SELinux 介绍

编译说明	Android SELinux	Yocto SELinux
是否支持单编	是	是
单编说明	针对image进行单编	针对所有target sepolicy编译
编译产物	生成 cil 文件或者 *.contexts 文件， mount 后可 push 文件验证	编译生成 policy.33， 二进制文件不可直接动态修改验证， remount 后也可 push 文件验证

5) Neverallow 差异

Android 上 SELinux 经过多年迭代， 加了很多 Neverallow 限制来管控 Google 认为敏感的权限、或者过大的权限； Yocto SELinux 相关的定义则比较少， 权限加起来相对容易， 没有 Android 卡得那么细， 但胜过 apparmor/smack 等方案。